

使用 Megatron Core 可扩展训练混合专家模型

技术报告 · NVIDIA · 2026-03-11

摘要

扩展混合专家 (MoE) 训练引入了密集模型中不存在的系统性挑战。由于每个 token 只激活一部分专家，这种稀疏性使得总参数数量的增长远快于每个 token 的计算量，在内存、通信和计算三个维度上形成耦合约束——优化其中一个维度往往会将压力转移到另一个，需要跨整个系统栈进行协同设计。

我们通过涵盖内存（细粒度重计算、卸载等）、通信（优化分发器、重叠等）和计算（分组 GEMM、算子融合、CUDA Graphs 等）的集成优化来应对这些挑战。该框架还提供了 **Parallel Folding**（并行折叠）以支持灵活的多维并行，以及 FP8/NVFP4 低精度训练支持和高效长上下文训练。在 NVIDIA GB300 和 GB200 上，DeepSeek-V3-685B 实现了 **1233/1048 TFLOPS/GPU**，Qwen3-235B 实现了 **974/919 TFLOPS/GPU**。

作为高性能、可扩展、生产就绪的开源解决方案，它已被学术界和工业界广泛用于在规模达数千 GPU 的集群上训练从数十亿到数万亿参数的 MoE 模型。

1. 引言

在大规模训练密集 Transformer 时，计算量随模型大小线性增长。MoE 模型遵循不同的扩展规律：通过将每个 token 路由到所选的专家子集而非激活所有参数，每个 token 的计算量随模型大小亚线性增长。近期的 MoE 模型在与质量匹配的密集模型相比，训练成本降低了一个数量级。

然而，大规模训练 MoE 引入了密集模型框架无法应对的系统挑战。本报告介绍 **Megatron-Core MoE**——Megatron-Core 中的 MoE 训练栈，涵盖训练万亿参数级 MoE 模型所需的架构、并行策略和系统优化。

1.1 混合专家模型 (MoE)

MoE 模型通过一组专用子网络（称为“专家”）以及一个轻量级路由器（门控网络）来增强标准神经网络，路由器动态选择哪些专家处理每个输入。在基于 Transformer 的语言模型中，MoE 层通常替换密集的前馈网络 (FFN) 块：不再对所有 token 使用单一 FFN，而是 MoE 层包含多个 FFN 专家，每个 token 根据学习到的路由权重被路由到其中一个小子集（例如 top-k）。

形式化定义：给定输入 token 表示 x ，路由器计算 E 个专家上的概率分布：

$$p(x) = \text{Softmax}(W_r \cdot x)$$

MoE 层的输出是所选专家输出的加权组合：

$$\text{MoE}(x) = \sum_{\{i \in \text{TopK}(p(x))\}} p_i(x) \cdot E_i(x)$$

该架构具有三大优势：

- **可扩展容量**：通过增加专家来独立于计算成本地增长模型容量
- **计算效率**：每个 token 只激活部分参数，相比等规模密集模型减少 FLOPs
- **专业化**：不同专家可以在不同输入类型上实现专业化

MoE 的发展历程：GShard 开创了大规模分布式 MoE 训练，引入了专家并行（EP）和负载均衡辅助损失；Switch Transformer 证明 MoE 可扩展至万亿参数同时保持训练稳定性；GLaM 证明 MoE 可以以密集模型训练成本的一小部分达到相同质量；Mixtral-8x7B 证明开放权重 MoE 模型可匹配专有密集模型；DeepSeek-V2/V3 引入了细粒度专家架构，使用数百个小专家最大化容量与计算比。

1.2 训练大规模 MoE 模型的挑战

随着模型向数百个专家、更小单个容量推进，MoE 训练的系统挑战也成比例增长。这些挑战源于一个根本原因：**MoE 的稀疏性**——表现为参数-计算失配（总参数远超激活计算量）和密集-稀疏失配（注意力层和 MoE 层需要相互冲突的并行配置）。

三堵墙（Three Walls）：

内存墙（Memory Wall）

所有 E 个专家的参数、梯度和优化器状态在训练期间必须驻留在内存中，即使每个 token 只激活 K 个。以 DeepSeek-V3 为例：685B 总参数，每 token 只有 37B 被激活，18x 的差距，造成远超等计算量密集模型的内存压力。减轻压力需要在别处付出代价：跨更多设备分发参数消耗通信带宽；重计算激活而非存储会消耗额外计算；卸载到主机内存消耗 PCIe 带宽。

通信墙（Communication Wall）

EP 需要 all-to-all 集合通信来将 token 分发到其分配的专家并收集结果。每次 all-to-all 的每 GPU 发送量约为 $T \cdot K \cdot h \cdot (EP-1)/EP$ ，完整的分发+合并循环使这个量翻倍。随着 EP 增大，通信越来越多地从高带宽节点内链路（如 NVLink）转移到较窄的节点间互连，可用带宽下降一个数量级。在 DeepSeek-V3 等架构中，未优化的 all-to-all 可消耗高达 60% 的总训练时间。

计算效率墙（Compute Efficiency Wall）

- **小 GEMM**：细粒度专家产生许多小矩阵乘法，GPU 计算单元利用率低
- **路由和排列开销**：token 路由和排列在优化后仍增加约 9% 的层执行时间
- **负载不均衡**：动态路由给专家分配不均匀的 token 数量，造成计算容量浪费
- **主机开销**：MoE 启动的内核数量更多，每次启动都有固定的主机侧开销

1.3 Megatron-Core MoE

构建于基于 PyTorch 的大规模 Transformer 训练库 Megatron-Core 之上，该 MoE 训练栈同时应对

三堵墙：

- **多维并行**：EP 与张量并行、流水线并行、序列并行和数据并行无缝集成。MoE Parallel Folding 解耦注意力和 MoE 层配置，打破传统 $EP \leq DP$ 的约束
- **内存优化**：细粒度激活重计算、内存高效排列、精度感知优化器和激活卸载减少内存占用
- **通信优化**：高性能 token 分发器 (DeepEP、HybridEP)，通信-计算重叠隐藏 all-to-all 延迟
- **计算优化**：分组 GEMM 内核、算子融合、CUDA Graphs 和同步无关执行
- **生产特性**：负载均衡策略、token 丢弃与容量控制、分布式优化器和 FSDP 支持、分布式检查点、从密集检查点 upcycling 等

1.4 本报告结构

- **第2节**：Megatron-Core MoE 架构（路由器、token 分发器、专家及前向传递）
- **第3节**：Parallel Folding 与多维并行（解决密集-稀疏失配问题）
- **第4节**：突破三堵墙（内存、通信、计算效率）
- **第5节**：FP8/FP4 低精度训练（跨越三堵墙的交叉优化）
- **第6节**：长上下文 MoE 训练（注意力主导时的优化策略）
- **第7节**：生产特性
- **第8节**：性能评估
- **第9节**：性能最佳实践与 DeepSeek-V3 案例研究
- **第10节**：Megatron-Core MoE 在强化学习中的应用